

Week 37: Gradient Descent, Momentum and Automatic Differentiation

First: cross-validation done properly. Then: finding the minimum when there is no closed form

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 37

Monday lecture (2×45 min):

- ▶ **First 30 min:** cross-validation, once more and properly – the k -fold loop, the leak, and the selection recipe of Section 3.14 (grid, standard error, one-SE rule, refit, test set once). Last week we ran out of time here, and Project 1 parts d) and i) need it.
- ▶ Why we must optimise numerically; convexity and Newton's method (Sections 4.1–4.2)
- ▶ Plain gradient descent, applied to OLS and Ridge where we know the answer (Sections 4.3–4.4)
- ▶ The learning rate and the condition number: the one number that governs everything (Section 4.5)
- ▶ Momentum (Section 4.6)
- ▶ Automatic differentiation: the gradient without deriving it, with JAX applied to the same problems (Section 4.14)

Tuesday session (2×45 min, flipped): your questions, then two cases – your own gradient descent code for OLS and Ridge (analytical gradients, learning rate, stopping), and momentum plus automatic differentiation on the Runge function of Project 1. Bring your laptop.

Reading

Chapter 3, Section 3.14 and Chapter 4, Sections 4.1–4.6 and 4.14 of the book. Goodfellow et al., Chapter 4 and Sections 8.1–8.3; Hastie et al., Section 7.10. Week 38: stochastic gradient descent, AdaGrad, RMSProp and Adam, with Lasso and logistic regression as applications.

Cross-validation: structuring the split (Section 2.12)

A single train/test split wastes data, and the answer depends on which points landed where.

k -fold cross-validation removes the imbalance:

1. Shuffle the data set at random.
2. Split it into k roughly equal, mutually exclusive folds.
3. Each fold in turn is the test set: fit on the other $k - 1$ folds, evaluate on the held-out fold, record the score, discard the model.
4. Summarise by the mean – and the spread – of the k scores.

Every observation is tested exactly once and trained on exactly $k - 1$ times. $k = n$ is leave-one-out; k between 5 and 10 is the usual compromise.

What it is for

For each λ on a logarithmic grid, and each fold i , fit $\theta_{-i}(\lambda)$ on the data with fold i removed, Eq. (2.54), score on fold i , average over folds; choose the λ minimising the cross-validated error; *refit* on all training data. The same recipe selects a polynomial degree, a Lasso penalty, a learning rate, a network width. **The one iron rule: the test set is touched exactly once, at the very end.**

Code: own KFold loop vs cross_val_score (Tuesday of week 36)

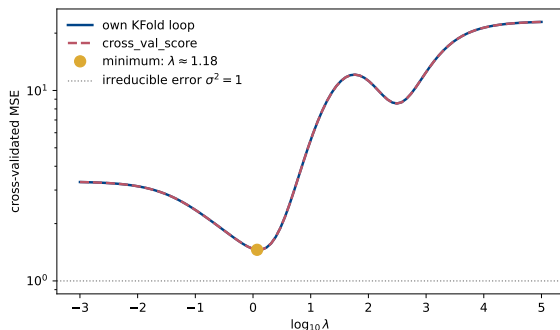
```
rng = np.random.default_rng(2026)
x = rng.standard_normal(100)
y = 3*x**2 + rng.standard_normal(100) # noise variance is 1

lambdas = np.logspace(-3, 5, 100)
kfold = KFold(n_splits=5, shuffle=True, random_state=2026)

# own loop: scaler and Ridge fitted on the training folds only
for i, lmb in enumerate(lambdas):
    for j, (train_inds, test_inds) in enumerate(kfold.split(x)):
        Xtrain = poly.fit_transform(x[train_inds][:, np.newaxis])
        Xtest = poly.transform(x[test_inds][:, np.newaxis])
        scaler = StandardScaler().fit(Xtrain)
        ridge = Ridge(alpha=lmb).fit(scaler.transform(Xtrain), y[train_inds])
        ypred = ridge.predict(scaler.transform(Xtest))
        scores_KFold[i, j] = np.mean((ypred - y[test_inds])**2)

# the same in three lines
pipe = make_pipeline(PolynomialFeatures(degree=6), StandardScaler(), Ridge(alpha=lmb))
scores = cross_val_score(pipe, x[:, np.newaxis], y, cv=kfold,
                          scoring="neg_mean_squared_error")
```

Reading the cross-validation curve



Degree-6 polynomial on $y = 3x^2 + \varepsilon$,
 $n = 100$, 5 folds.

- ▶ The two curves lie on top of each other: `cross_val_score` does nothing magical.
- ▶ The minimum sits near $\lambda \approx 1$ with CV-MSE ≈ 1.5 , just above the irreducible $\sigma^2 = 1$: the method has recovered the noise floor from 100 points.
- ▶ At $\lambda = 10^5$ every coefficient is crushed: the model predicts the mean and the error climbs to $\text{Var}(y)$.

The `StandardScaler` *inside* the fold loop is not decoration: a degree-6 basis has columns differing by orders of magnitude, and without scaling the curve tails are numerical noise. We will meet the same columns again today, as the reason gradient descent is slow.

Pause and think 1: the subtle leak

Question

A pipeline scales the full data set *once*, then runs 5-fold cross-validation on the scaled data. The CV error comes out slightly – but systematically – too optimistic. Why?

Pause and think 1: the subtle leak

Question

A pipeline scales the full data set *once*, then runs 5-fold cross-validation on the scaled data. The CV error comes out slightly – but systematically – too optimistic. Why?

Answer

The scaler's mean and variance were computed on *all* the data, so each training fold has already seen statistics of its own test fold: a leak. Small here, large when features are many. Rule: **every** preprocessing step that learns from data – centring, scaling, feature selection, PCA, imputation – goes *inside* the loop, refitted per fold. In `scikit-learn`: wrap it in a `Pipeline` and cross-validate the pipeline. The same discipline applies to the bootstrap.

Three errors, three roles (Section 3.14)

- ▶ **Model parameters** θ_0, θ : fixed, at given λ , by minimising the cost – the training error.
- ▶ **Hyperparameter** λ : cannot be fixed the same way. The unpenalised fit minimises the training error over *all* θ , so for every $\lambda > 0$

$$\text{Err}_{\text{Train}}(\lambda) \geq \text{Err}_{\text{Train}}(0), \quad \text{Eq. (3.91),}$$

and the training error is non-decreasing along the whole path. Minimising it over λ always returns $\lambda = 0$.

The sentence to remember

The **training error** is minimised to get the parameters at fixed λ ; the **cross-validation error** is minimised over λ to get the hyperparameter; the **test error** is computed once, at the end, and minimised over nothing.

Later today the first of these three becomes a numerical problem: minimising the training error over θ when no closed form exists.

The procedure, precisely

Assign each of the n training points to a fold, $\kappa(i) \in \{1, \dots, K\}$, after a shuffle. With $\hat{\theta}_0^{(-k)}(\lambda)$, $\hat{\boldsymbol{\theta}}^{(-k)}(\lambda)$ fitted – *scaler included* – to the points not in fold k ,

$$\text{CV}_K(\lambda) = \frac{1}{n} \sum_{i=0}^{n-1} \left(y_i - \hat{\theta}_0^{(-\kappa(i))}(\lambda) - \mathbf{x}_i^T \hat{\boldsymbol{\theta}}^{(-\kappa(i))}(\lambda) \right)^2 = \frac{1}{K} \sum_{k=1}^K \text{MSE}_k(\lambda), \quad \text{Eq. (3.92),}$$

every point predicted once, by the model that did not see it. The spread of the K fold errors gives a standard error, Eq. (3.93),

$$\text{SE}_K(\lambda) = \frac{1}{\sqrt{K}} \text{std}(\text{MSE}_1, \dots, \text{MSE}_K)$$

– a rough guide, not a confidence interval, since the K training sets overlap. Then $\lambda_{\min} = \arg \min_{\lambda} \text{CV}_K(\lambda)$, Eq. (3.94).

The two steps people forget

Refit: the K sets of fold coefficients were temporary – the final model is a *new* fit at $\lambda_{\min}$ to all n training points, scaler refitted on all n . **Test once:** only then, and only once, the test set. Report λ , the coefficients, $\text{CV} \pm \text{SE}$, and the test error.

Which λ is being compared? The factor n

The fold models see $n(K - 1)/K$ points, the final one n . Is the same numerical λ the same penalty? Depends on the convention.

$$\frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2 \Rightarrow \hat{\theta} = (X^T X + n\lambda I)^{-1} X^T y \quad \text{Eq. (3.95)}$$

$X^T X$ grows like n , and so does $n\lambda$: λ is a penalty *per observation* and transfers from 80-point folds to the 100-point refit. Without the $1/n$ – Eq. (3.44), $(X^T X + \lambda I)^{-1}$ – the same λ is weaker by $(K - 1)/K$ on the full set.

The library conventions (Section 3.10)

scikit-learn's Ridge: no $1/n$, so $\alpha = n\lambda$. Its Lasso: $\|\cdot\|^2/(2n) + \alpha \|\theta\|_1$, so $\alpha = \lambda/2$. The book keeps the $1/n$ throughout, and so should your own code – **including the gradient descent code you write this week**: the same $1/n$ decides what your gradient looks like and which closed form it must converge to.

The grid: logarithmic. Lasso: from $\lambda_{\max} = \frac{2}{n} \|X^T y\|_\infty$ (Eq. (3.67)) down three or four decades. Ridge: for standardised columns 10^{-4} to 10 covers every regime. A minimum at the end of the grid means the grid was wrong.

The one-standard-error rule

Near its minimum the CV curve is *flat*, and the minimum is located only to within the noise. Among penalties the folds cannot tell apart, take the most regularised:

$$\lambda_{1SE} = \max\left\{\lambda : CV_K(\lambda) \leq CV_K(\lambda_{\min}) + SE_K(\lambda_{\min})\right\}, \quad \text{Eq. (3.96).}$$

- ▶ For Ridge: a little more bias the data cannot detect, for a smaller variance.
- ▶ For the Lasso: a *sparser* model – usually why one uses the Lasso at all.
- ▶ A convention, not a theorem: the improvement of the more complex model is not larger than the uncertainty with which it is measured, so it is not believed.

How unstable is $\lambda_{\min}$?

Twenty different shuffles of the same 100 training points (book, Section 3.14): $\lambda_{\min}$ moves by a factor 5 for Ridge and by more than an order of magnitude for the Lasso, taking the Lasso's support with it. The prediction is stable; the hyperparameter and the selected variables are not. Remedies: repeated CV (costly), the one-SE rule (cheap), the bootstrap for the support.

Pause and think 2: the second decimal of $\lambda_{\min}$

Question

Two students run 5-fold CV for Ridge on the same 100 points with different shuffles and report $\lambda_{\min} = 0.003$ and 0.013 . Who is right, and does it matter?

Pause and think 2: the second decimal of $\lambda_{\min}$

Question

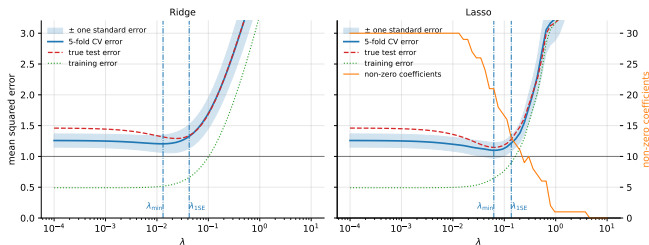
Two students run 5-fold CV for Ridge on the same 100 points with different shuffles and report $\lambda_{\min} = 0.003$ and 0.013 . Who is right, and does it matter?

Answer

Both, and mostly no. The CV curve is flat along that whole valley: its width there is smaller than the standard error, so the folds cannot rank the two values, and the test errors of the two refitted models differ by about a tenth. What *does* change is the fitted model – and for the Lasso, which variables it keeps. Report $\lambda_{\min}$ with its *range*, or use the one-SE rule, which moves the choice onto the rising flank where the curve determines its own position.

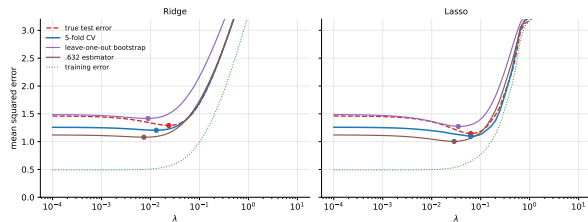
Ridge and Lasso side by side: a sparse truth (book Figure 3.4)

$p = 30$ correlated features (corr. $0.7^{|i-j|}$), 8 true non-zeros, $\sigma = 1$, 100 training points. Dashed: the *true* test error of the refit (20 000 fresh points – never available in practice).



- ▶ Training error (dotted): monotone in λ , far below the test error – it can select nothing.
- ▶ Ridge: $\lambda_{\min} = 0.013$, true optimum 0.024; the true errors differ by 0.02. The choice costs nothing.
- ▶ Lasso: $\lambda_{\min}$ hits the true optimum; true error 1.15 vs Ridge 1.29 – the truth is sparse.
- ▶ But the Lasso keeps 21 of 30 (8 true + 13 spurious): CV optimises *prediction*, and small false coefficients cost nothing. The one-SE choice keeps 13, all 8 true ones still in, at 1.28.

Cross-validation versus the bootstrap along the grid (book Figure 3.5)



A bootstrap replica misses a point with probability $(1 - 1/n)^n \rightarrow e^{-1}$, Eq. (3.97), so it holds 63.2% of the distinct points. The leave-one-out bootstrap, Eq. (3.98), is pessimistic; the .632 estimator, Eq. (3.99), averages the two biases and is the best score at fixed λ . Yet cross-validation selects:

1. **Location, not level.** CV's bias is nearly constant in λ ; the .632 estimate inherits 0.368 of the training error and is dragged towards under-regularisation.
2. **Cost.** K fits per grid point against $B \sim 200$.
3. **Sample size.** Bootstrap models see 63% of the points and fall off the $p \approx n$ cliff first.
4. **No correction constants.**

CV chooses the penalty; the bootstrap says which coefficients to believe (refit at $\lambda_{\min}$ to B replicas, count the non-zeros).

The recipe in one place – and the one thing not to report

1. Set aside a test set; do not look at it until step 6.
2. Logarithmic grid of penalties.
3. Shuffle the training data, K folds, K between 5 and 10.
4. For every fold and every λ : standardise with the training folds' statistics, fit, predict the held-out fold; average to get CV_K and SE_K .
5. Select $\lambda_{\min}$ or λ_{1SE} ; **refit** on all training data.
6. Evaluate on the test set, once. Report λ , coefficients, $CV \pm SE$, test error.

Do not report $CV_K(\lambda_{\min})$ as “the error of my model”

It is the minimum of sixty noisy estimates – biased downwards, the more so the more penalties you try. A test set answers the question; without one, nest the whole selection inside an outer K_{out} -fold loop. Negligible for linear models; essential for flexible models tuned on fine grids.

Cross-validation chooses the regularisation; refitting produces the final parameters; the test set is a test set only as long as it has been used once. This is what Project 1, parts d) and i), ask for.

Why we now need to optimise numerically

Every problem so far: a data set X , a model $g(\theta)$, a cost $C(X, g(\theta))$, and the parameters that minimise the cost.

- ▶ OLS and Ridge: the cost is quadratic, the gradient vanishes at a point we can write down, and the minimisation is a linear system, Eqs. (3.44) and (3.46).
- ▶ Lasso: convex but not differentiable – coordinate descent (Section 3.9). *Already* no closed form.
- ▶ Logistic regression (week 38, Chapter 5), neural networks (Chapter 8): no closed form, and for networks not even convexity.

The plan of Chapter 4, and of the next two weeks

We build the numerical tools *on OLS and Ridge*, precisely because we know the answer there and can measure every method against it. Today: plain gradient descent and momentum, and the one number that makes gradient descent slow – the condition number of the Hessian, which we met in Section 1.12 (numerically) and Section 3.7 (statistically). Week 38: stochastic gradients and the adaptive methods AdaGrad, RMSProp and Adam.

Gradient descent takes the same step in every direction, but the cost does not curve equally in every direction. The whole art lies in fixing that mismatch.

Convexity (Section 4.1)

A set $C \subset \mathbb{R}^n$ is **convex** if the segment between any two of its points lies in C . A function f on a convex set is **convex** if the chord lies above the graph,

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2), \quad t \in [0, 1], \quad \text{Eq. (4.1);}$$

strict inequality for $x_1 \neq x_2$, $t \in (0, 1)$: *strictly* convex.

First-order condition, Eq. (4.2)

Differentiable f is convex iff

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) \quad \forall x, y :$$

the tangent plane is a global underestimator.

Second-order condition

Twice differentiable f is convex iff its Hessian is positive semi-definite everywhere; in one variable $f'' \geq 0$. This is the *usable* one: a procedure, not a definition.

Why we care

Any stationary point of a convex function is a global minimum. Proof in one line from Eq. (4.2) with $x = x^*$, $\nabla f(x^*) = 0$: $f(y) \geq f(x^*)$ for every y . No local minima to be trapped in, no saddle points to be delayed by.

We have already used it twice

- ▶ **OLS.** $H = \frac{2}{n}X^T X$ (Section 1.8.5), and $z^T X^T X z = \|Xz\|_2^2 \geq 0$: positive semi-definite, so the cost is convex and the normal equations give *the* global minimum.
- ▶ **Ridge.** $H = \frac{2}{n}X^T X + 2\lambda I$, positive *definite* for $\lambda > 0$: strictly convex, unique minimum – even when $p > n$.
- ▶ **Lasso.** Convex but not differentiable at $\theta_j = 0$, which is why Section 3.9 needed coordinate descent (and subgradients) rather than a gradient.

The bad news, said once

The cost functions of the later chapters are not convex. A network with one hidden layer has a cost surface with many local minima, and the theorem does not apply. We use the methods of this chapter there anyway, because they work in practice – relying on empirical good behaviour, not on a theorem. Know which of the two you are relying on.

Pause and think 3: convex, but how many minima?

Question

For OLS with $p > n$ (more features than data points) the Hessian $\frac{2}{n}X^T X$ is singular. Is the cost still convex? Is the minimum unique? What does gradient descent converge to?

Pause and think 3: convex, but how many minima?

Question

For OLS with $p > n$ (more features than data points) the Hessian $\frac{2}{n}X^T X$ is singular. Is the cost still convex? Is the minimum unique? What does gradient descent converge to?

Answer

Convex, yes: positive *semi*-definite is all Eq. (4.1) needs. Unique, no: every $\theta^* + z$ with $Xz = 0$ has the same cost, a whole flat valley of global minima. Gradient descent started from 0 never moves along the null space (the gradient $\frac{2}{n}X^T(\cdot)$ lives in the row space of X), so it converges to the *minimum-norm* solution – the same one the pseudoinverse of Section 1.19 picks. Adding any $\lambda > 0$ removes the valley: that is the geometric content of “Ridge makes the problem strictly convex”.

Newton's method: the ideal we shall approximate (Section 4.2)

Root finding. Taylor expand f about x near the root s , Eq. (4.3), keep the linear term: $s \approx x - f(x)/f'(x)$, hence the iteration

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad \text{Eq. (4.4) :}$$

follow the tangent to the axis. Quadratic convergence near the root; can fail completely far from it, or if f' nearly vanishes. In several variables the derivative becomes the Jacobian and the step $h = -J^{-1}f$, Eq. (4.7). **Minimisation** means a root of ∇C ; the Jacobian of the gradient is the Hessian:

$$\theta_{k+1} = \theta_k - H^{-1}(\theta_k) \nabla C(\theta_k) \quad \text{Eq. (4.8).}$$

This minimises the second-order Taylor expansion of the cost *exactly*. Two properties no gradient-only method can match: quadratic convergence near the minimum, and *invariance under linear rescaling* of the parameters – the conditioning problems of the rest of today simply do not arise.

Newton on OLS: one step, exactly – and why we do not use it

With $\nabla C = \frac{2}{n}X^T(X\theta - y)$ and $H = \frac{2}{n}X^TX$, one step from *any* start:

$$\theta_1 = \theta_0 + (X^TX)^{-1}X^T(y - X\theta_0) = (X^TX)^{-1}X^Ty, \quad \text{Eq. (4.9).}$$

Not a coincidence: for a quadratic cost the second-order expansion *is* the cost, so one Newton step lands on the minimum. The normal equations are Newton's method in disguise.

The price

p parameters: the Hessian has p^2 entries and inverting it costs $\mathcal{O}(p^3)$ (Section 1.14). Twenty features: nothing. A network with 10^7 parameters: 10^{14} numbers, which cannot be stored, let alone factorised. **The whole of Chapter 4 constructs methods that capture some of Eq. (4.8) at a cost *linear* in p .**

Keep Eq. (4.8) in view all day: every method we meet replaces H^{-1} by something cheaper – a scalar η (gradient descent), a scalar plus memory (momentum), a diagonal built from past gradients (week 38's AdaGrad, RMSProp, Adam).

Gradient descent (Section 4.3)

A function decreases fastest, locally, along $-\nabla F(x)$. For

$$x_{k+1} = x_k - \gamma_k \nabla F(x_k), \quad \gamma_k > 0, \quad \text{Eq. (4.10),}$$

and γ_k small enough, $F(x_{k+1}) \leq F(x_k)$. Iterating from x_0 : **steepest descent**, or **gradient descent** (GD). The step length γ_k is the **learning rate**, from now on η . If F is convex, the point we reach is the global minimum.

Limitations, before we fall in love with it

- ▶ Deterministic: on a non-convex surface it descends into whichever local minimum it faces; the result depends on x_0 .
- ▶ The full gradient is a function of all p parameters and *all* n data points – expensive (week 38: stochastic gradients).
- ▶ Delicate in η : too large diverges, too small crawls. Choosing η is *the* practical difficulty, and Sections 4.9–4.11 exist to remove the need to do it by hand.

Gradient descent for linear regression (Section 4.4)

The ideal test case: an analytical solution to check against, an exact gradient, and a convex cost so convergence is guaranteed for small η . Data $y_i = 4 + 3x_i + \varepsilon_i$, $x_i \sim \mathcal{U}(0, 2)$, $n = 100$, with an intercept column, Eq. (4.11):

$$C(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 = \frac{1}{n} \sum_i [(\theta_0 + \theta_1 x_i)^2 - 2y_i(\theta_0 + \theta_1 x_i) + y_i^2], \quad \text{Eq. (4.12).}$$

Differentiate coordinate by coordinate and reassemble:

$$\nabla_{\boldsymbol{\theta}} C = \frac{2}{n} \begin{bmatrix} \sum_i (\theta_0 + \theta_1 x_i - y_i) \\ \sum_i x_i (\theta_0 + \theta_1 x_i - y_i) \end{bmatrix} = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}), \quad \text{Eq. (4.13);} \quad \mathbf{H} = \frac{2}{n} \mathbf{X}^T \mathbf{X}, \quad \text{Eq. (4.14).}$$

The iteration,

$$\boxed{\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}_k)} \quad \text{Eq. (4.15),}$$

started from a random $\boldsymbol{\theta}_0$ and stopped when $\|\nabla C\| \leq \epsilon$, say 10^{-8} . This is Project 1, part e), in one line – the rest of part e) is about what η does.

Code: gradient descent against the closed form (Section 4.4)

```
n = 100
rng = np.random.default_rng(2024)
x = 2.0 * rng.random((n, 1))
y = 4.0 + 3.0 * x + rng.normal(size=(n, 1))
X = np.c_[np.ones((n, 1)), x]

theta_exact = np.linalg.pinv(X.T @ X) @ X.T @ y # Chapter 3
print("analytical:", theta_exact.ravel())

theta = rng.normal(size=(2, 1)) # Eq. (4.15)
eta = 0.1
for k in range(1000):
    gradient = (2.0 / n) * X.T @ (X @ theta - y) # Eq. (4.13)
    theta -= eta * gradient
    if np.linalg.norm(gradient) < 1.0e-8:
        break
print(f"gradient descent after {k+1} iterations:", theta.ravel())
```

With $\eta = 0.1$ the iteration reproduces the analytical answer to five decimals within a few hundred steps. With $\eta = 0.001$ – the value in the old lecture notes – a thousand iterations are not nearly enough; with η too large it diverges. **Section 4.5 says exactly where the boundary lies.**

Gradient descent for Ridge, and the factor n once more (Section 4.4.1)

$$C(\theta) = \frac{1}{n} \|X\theta - y\|_2^2 + \lambda \theta^T \theta, \quad \text{Eq. (4.16);} \quad \nabla C = \frac{2}{n} X^T (X\theta - y) + 2\lambda \theta, \quad H = \frac{2}{n} X^T X + 2\lambda I, \quad \text{Eq. (4.17)}$$

Setting the gradient to zero returns the closed form – *with the n .*

```
lambda = 0.001
theta = rng.normal(size=(2, 1))
for k in range(1000):
    gradient = 2.0 * (X.T @ (X @ theta - y) / n + lambda * theta) # Eq. (4.17)
    theta -= eta * gradient

I = np.eye(X.shape[1])
theta_exact = np.linalg.inv(X.T @ X + n * lambda * I) @ X.T @ y # Eq. (3.95)
print("gradient descent:", theta.ravel())
print("closed form: ", theta_exact.ravel())
```

Our cost carries $1/n$ on the data term but not on the penalty, so the closed form must be $(X^T X + n\lambda I)^{-1} X^T y$. Drop the n and your “converged” gradient descent disagrees with your “exact” answer – and both are right, for different λ . This is the same bookkeeping as Eq. (3.95) this morning.

Pause and think 4: same λ , two answers

Question

A student's Ridge gradient descent converges beautifully – the gradient norm goes to 10^{-10} – but the parameters disagree with `sklearn.linear_model.Ridge(alpha=lmbda)` in the second decimal. Which of the two is wrong?

Pause and think 4: same λ , two answers

Question

A student's Ridge gradient descent converges beautifully – the gradient norm goes to 10^{-10} – but the parameters disagree with `sklearn.linear_model.Ridge(alpha=lmbda)` in the second decimal. Which of the two is wrong?

Answer

Neither. Ridge minimises $\|y - X\theta\|^2 + \alpha\|\theta\|^2$ with no $1/n$, so its alpha equals the book's $n\lambda$; the student's cost, Eq. (4.16), has the $1/n$. Converged gradient descent at λ equals `Ridge(alpha=n*lmbda)` – and also the closed form with $n\lambda$. Before comparing methods, compare cost functions. (Also check that both centred y and left the intercept unpenalised, Section 3.13.)

The learning rate and the condition number (Section 4.5)

For a quadratic cost with Hessian H (OLS, Ridge), write the error $e_k = \theta_k - \theta^*$. Since $\nabla C = He$, Eq. (4.15) gives

$$e_{k+1} = e_k - \eta He_k = (I - \eta H)e_k, \quad \text{Eq. (4.18).}$$

Diagonalise, $H = Q\Lambda Q^T$ (Section 1.9), and express the error in the eigenbasis, $\tilde{e} = Q^T e$. The recursion *decouples completely*:

$\tilde{e}_{k,i} = (1 - \eta\lambda_i)^k \tilde{e}_{0,i}$

 Eq. (4.19).

Each eigendirection contracts by its own factor $|1 - \eta\lambda_i|$ per step. **This single equation contains everything.**

Stability, Eq. (4.20)

All directions converge iff $|1 - \eta\lambda_i| < 1$ for all i :

$$0 < \eta < \frac{2}{\lambda_{\max}}.$$

Set by the *steepest* direction, whatever the others do.

Speed, Eq. (4.21)

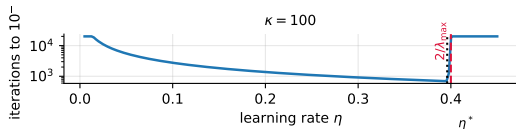
The slowest direction is $\lambda_{\min}$. Best $\eta^* = 2/(\lambda_{\max} + \lambda_{\min})$, worst contraction factor

$$\frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \kappa_2(H).$$

Iterations to a fixed accuracy $\propto \kappa$.

The central result of the chapter, in a picture (book Figure 4.1)

Quadratic with eigenvalues $\{0.05, 1, 5\}$: iterations to an error of 10^{-6} against η . The count falls, reaches its minimum at the predicted η^* , rises, and diverges *exactly* at $2/\lambda_{\max}$. No margin of safety: fastest convergence and outright divergence are a factor two apart in η .



Read it slowly

Gradient descent is not slow because gradients are a bad idea. It is slow because it applies *the same* η to every eigendirection, while stability forces that η to be dictated by the steepest one. With $\kappa = 1000$ the step that is barely stable along the steep direction is a thousand times too small along the flat one – and the flat one is where the remaining error lives. The iterates oscillate across the valley while creeping along its floor.

What this means for regression (Section 4.5)

For least squares $H = \frac{2}{n}X^T X$, so

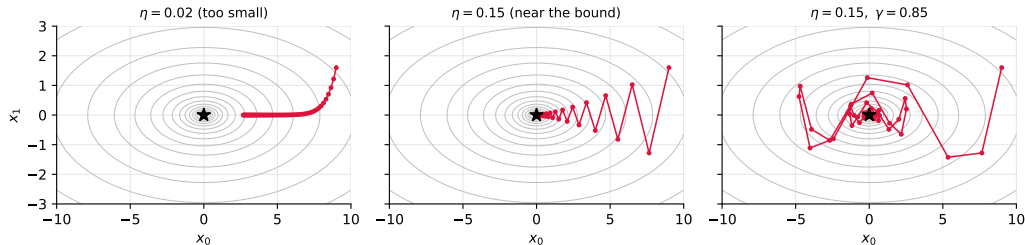
$$\kappa(H) = \kappa_2(X^T X) = \kappa_2(X)^2, \quad \text{Eq. (4.22) :}$$

the iteration count scales with the *square* of the condition number of the design matrix. Three consequences, each connecting backwards:

1. **Standardise the features.** Not only the statistical convention of Section 3.13 but a direct accelerator: comparable columns, smaller κ , fewer iterations. A raw degree-6 polynomial basis on $[-3, 3]$ has columns from 10^0 to 10^3 – *that* is why the unscaled CV curve tails were noise this morning, and why unscaled gradient descent does not move.
2. **Ridge improves conditioning as well as variance.** By Eq. (4.17) the Hessian is $2(X^T X/n + \lambda I)$ with condition number $(\lambda_{\max} + \lambda)/(\lambda_{\min} + \lambda) < \kappa$ for every $\lambda > 0$. Regularisation makes the optimisation easier.
3. **Conjugate gradients** (Section 1.16) achieve a rate governed by $\sqrt{\kappa}$ and are the right tool for *quadratic* costs at scale – but rely on exact gradients and a quadratic, and cope badly with the stochastic, non-convex problems ahead. Machine learning built its own remedies instead.

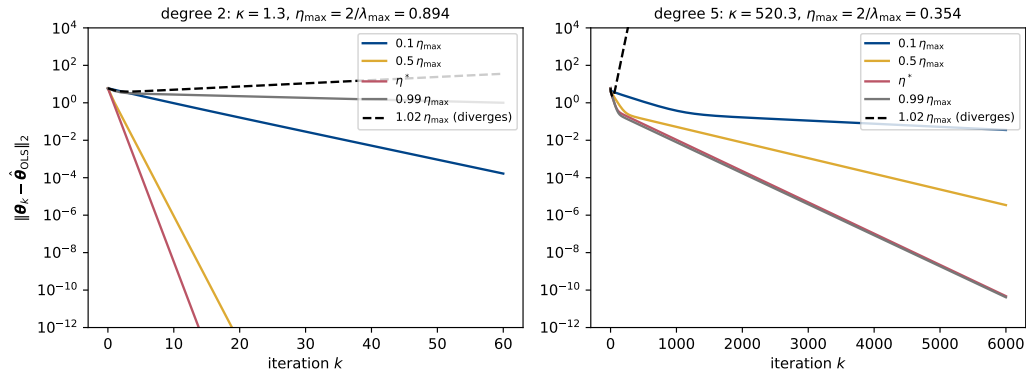
The algebra in pictures (book Figure 4.2)

$\kappa = 12$; stability bound $2/\lambda_{\max} = 0.167$



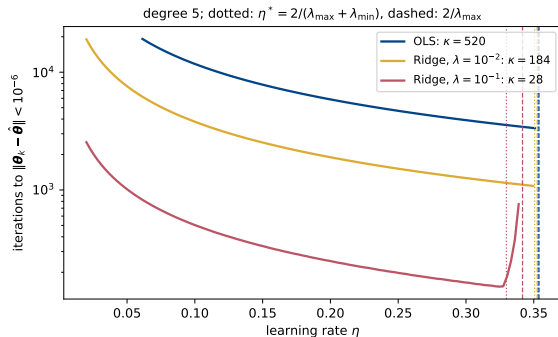
An elongated quadratic with $\kappa = 12$. **Left:** too small a step creeps along the valley floor. **Centre:** a step near the stability bound $2/\lambda_{\max}$ oscillates across the narrow direction while making slow progress along it. **Right:** the same step *with momentum*, $\gamma = 0.85$: the oscillation is cancelled and the drift along the flat direction accumulates. Sections 4.5 and 4.6 in one figure.

Our own numbers: this week's exercise data



$f(x) = 2 - x + 5x^2$ on $[-2, 2]$, $n = 100$, noise $\sigma = 0.5$, standardised polynomial columns, centred y (the week-37 exercises). **Left, degree 2:** x and x^2 are nearly uncorrelated on a symmetric interval, eigenvalues 1.76 and 2.24, $\kappa = 1.3$: at $\eta^* = 0.5$ the closed form $(-1.077, 5.855)$ is reached to 10^{-8} in 11 iterations; at $\eta = 0.01$, 1158. **Right, degree 5:** the odd powers x, x^3, x^5 are strongly correlated, eigenvalues from 0.011 to 5.65, $\kappa = 520$: the best learning rate needs 4964 iterations, $\eta = 0.1$ needs 11 836, and $\eta = 0.01$ has not converged after 10^5 . In both panels $1.02 \eta_{\max}$ diverges – the boundary of Eq. (4.20) is sharp.

Scanning the learning rate, and what Ridge does to κ



Degree-5 exercise data, iterations to $\|\theta_k - \hat{\theta}\| < 10^{-6}$.

- ▶ OLS: $\kappa = 520$; the minimum sits at $\eta^* = 0.353$, and 0.354 is the edge of the cliff.
- ▶ Ridge, $\lambda = 10^{-2}$: $\lambda_{\min}$ goes from 0.011 to 0.031, $\kappa = 184$, three times fewer iterations at the optimum.
- ▶ Ridge, $\lambda = 10^{-1}$: $\kappa = 28$, twenty times fewer.

The penalty lifts the *small* eigenvalues – the flat directions that gradient descent crawls along – and barely touches the large one, so $\eta_{\max}$ hardly moves while η^* does. Regularisation is good for the optimiser, not only for the estimator. Tuesday, Case 1: you produce both figures with your own code and check the eigenvalue bound on your own Hessian.

Pause and think 5: the $\eta = 0.001$ of the old notes

Question

Last year's notes ran gradient descent for OLS with $\eta = 0.001$ and 1000 iterations on the degree-2 exercise data and reported the result as “converged”. How far from the closed form was it, roughly?

Pause and think 5: the $\eta = 0.001$ of the old notes

Question

Last year's notes ran gradient descent for OLS with $\eta = 0.001$ and 1000 iterations on the degree-2 exercise data and reported the result as “converged”. How far from the closed form was it, roughly?

Answer

Not converged at all. With $\lambda_{\min} = 1.76$ the slow direction contracts by $(1 - 0.001 \cdot 1.76)^{1000} = e^{-1.76} \approx 0.17$ per thousand iterations: after 1000 steps a sixth of the initial error is still there, in the *second* digit of θ . A converged run needs $\|\nabla C\|$ below tolerance, not a fixed iteration count. And the converse trap: on this problem $\eta = 0.8$ is fine, but move to degree 5 and $\eta_{\max} = 0.354$ – the safe η is a property of the *data*, not of the algorithm. Compute $\lambda_{\max}$ of your Hessian; it is one line.

Momentum: give the iteration a memory (Section 4.6)

Replace Eq. (4.10) by

$$\boxed{v_{k+1} = \gamma v_k + \eta \nabla C(\theta_k), \quad \theta_{k+1} = \theta_k - v_{k+1}} \quad \text{Eq. (4.23),}$$

with $\gamma \in [0, 1)$ the **momentum parameter**, typically 0.9. Unrolling,

$$v_{k+1} = \eta \sum_{j=0}^k \gamma^{k-j} \nabla C(\theta_j), \quad \text{Eq. (4.24) :}$$

an exponentially weighted sum of *all* gradients seen so far; $\gamma = 0$ is plain gradient descent.

Flat direction, $\eta \lambda_i \ll 1$

Successive gradients point the same way and add up.
A constant gradient g accumulates to

$$v_{\infty} = \eta g \sum_j \gamma^j = \frac{\eta g}{1 - \gamma}, \quad \text{Eq. (4.25) :}$$

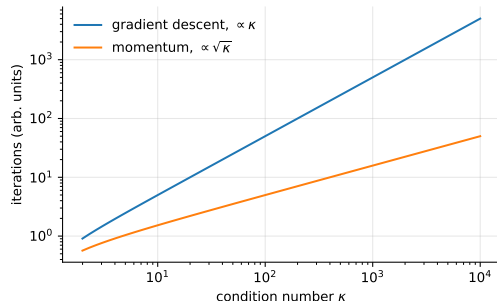
the effective step is multiplied by $1/(1 - \gamma)$ – ten for $\gamma = 0.9$.

Momentum amplifies the consistent and cancels the oscillatory – the right treatment for exactly the landscape Eq. (4.19) diagnosed.

Steep direction

The iterate overshoots, the gradient flips sign every step, successive terms in Eq. (4.24) alternate and largely *cancel*: the accumulated step is *smaller* than the bare gradient. The oscillation across the valley is damped.

From κ to $\sqrt{\kappa}$ (book Figure 4.3)



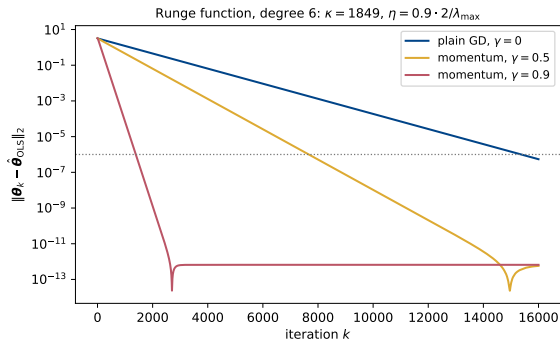
Along an eigendirection the momentum iteration is a *two-term* recurrence in $\tilde{\epsilon}$. With optimally chosen η and γ its contraction rate improves from Eq. (4.21) to

$$\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}, \quad \text{Eq. (4.26) :}$$

iterations scale with $\sqrt{\kappa}$ instead of κ . At $\kappa = 10^2$ an order of magnitude saved; at 10^4 , two. And by Eq. (4.22) the least-squares κ is the condition number of *X squared*, so these values are ordinary. $\sqrt{\kappa}$ is also the rate of conjugate gradients (Section 1.16): both build the step from the current gradient and the previous direction. Momentum is conjugate gradients with the optimal coefficients replaced by a fixed constant that needs no recomputation and no quadratic cost. **Nesterov momentum** evaluates the gradient at the look-ahead point $\theta_k - \gamma v_k$: if the velocity is about to overshoot, the gradient there already points back. A flag on every framework's optimiser.

Momentum on the Runge function – Project 1, parts e) and f)

$f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$, $n = 100$,
 $\sigma = 0.1$, degree 6, standardised columns:
 $\lambda_{\max} = 6.0$, $\lambda_{\min} = 0.0032$, $\kappa = 1849$. Same
 $\eta = 0.9 \cdot 2/\lambda_{\max} = 0.30$ for all three.
Iterations to $\|\theta_k - \hat{\theta}_{\text{OLS}}\| < 10^{-6}$:



plain GD, $\gamma = 0$	15 374
momentum, $\gamma = 0.5$	7 669
momentum, $\gamma = 0.9$	1 391

A factor 11 from a single extra line of code – in line with Eq. (4.26), $\sqrt{1849} = 43$ being the ceiling with *tuned* η, γ . At degree 10, $\kappa \approx 2 \times 10^6$ and plain gradient descent does not converge in 10^5 iterations: this is why part f) of the project exists. Tuesday, Case 2: this figure, and its η -sensitivity, from your own code.

Code: momentum is one line more than gradient descent

```
def gradient(theta, X, y, lam=0.0):
    """Eqs. (4.13) and (4.17): gradient of  $(1/n)||X\theta - y||^2 + \text{lam}\theta^T\theta$ ."""
    n = len(y)
    return (2.0 / n) * X.T @ (X @ theta - y) + 2.0 * lam * theta

def momentum_descent(X, y, eta, gamma, lam=0.0, num_iters=5000, tol=1e-8):
    """Gradient descent with momentum, Eq. (4.23). gamma = 0 is plain gradient descent."""
    theta, v = np.zeros(X.shape[1]), np.zeros(X.shape[1])
    history = [theta.copy()]
    for t in range(num_iters):
        g = gradient(theta, X, y, lam)
        v = gamma * v + eta * g # the memory
        theta = theta - v
        history.append(theta.copy())
        if np.linalg.norm(g) < tol:
            break
    return np.array(history), t + 1
```

The update rule does not care where `gradient` comes from – an analytical formula, or (next) automatic differentiation. That separation is the structure of your Project 1 code: one function for the cost, one for its gradient, one per optimiser.

Pause and think 6: why not $\gamma = 0.99$?

Question

If $\gamma = 0.9$ multiplies the effective step by 10, why not take $\gamma = 0.99$ for a factor 100, or 0.999?

Pause and think 6: why not $\gamma = 0.99$?

Question

If $\gamma = 0.9$ multiplies the effective step by 10, why not take $\gamma = 0.99$ for a factor 100, or 0.999?

Answer

Two reasons. First, stability: the two-term recurrence converges only for $\eta\lambda_{\max} < 2(1 + \gamma)$ *and* the effective step $\eta/(1 - \gamma)$ along the flat directions must not itself overshoot – with $\gamma \rightarrow 1$ the iterate becomes a nearly frictionless ball that rolls through the minimum and back for ever.

Second, the optimum is *tied to* κ : the rate of Eq. (4.26) is attained at

$\gamma^* = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2$, which is 0.91 for the Runge problem ($\kappa = 1849$) and 0.67 for $\kappa = 100$. Past γ^* the iteration count *rises* again. Default 0.9; tune with cross-validation if it matters; never 0.999 blindly.

Every method today needs a gradient (Section 4.14)

For OLS we derived it by hand, Eq. (4.13). For a network with a dozen layers that is possible but error-prone; for a model under active development, impractical. **Automatic differentiation** (AD) solves the problem completely.

The idea

Every program, however complicated, executes a sequence of elementary operations – $+$, $\times$, $\exp$, $\log$, $\sin$ – each with a known derivative. Apply the chain rule to that sequence and you obtain derivatives of any order, **accurate to working precision**, at a cost only a small constant factor above the program itself. Both halves of that sentence are theorems.

Plan for the next fifteen minutes: what AD is *not* (symbolic, numerical); the evaluation trace; the two modes, forward and reverse, from the chain rule; the cheap gradient principle; JAX on this morning's problems. *Reverse-mode AD is what backpropagation is*. Chapter 8 will meet it again under that name.

Three ways to differentiate a program (Section 4.14.1)

Symbolic. Manipulates the *formula*; exact, but needs a single closed-form expression (no loops, no branches) and suffers expression swell. **Numerical.** Central difference, Eq. (4.43),

$$D_h f(x) = \frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6} f'''(x) + \dots$$

Truncation error $\propto h^2$ (Eq. 4.44) wants h small; rounding error $\epsilon_M |f|/h$ wants it large:

$$E(h) \approx \frac{h^2}{6} |f'''| + \frac{\epsilon_M |f|}{h}, \quad \text{Eq. (4.45).}$$

Minimising, Eq. (4.46):

$$h^* \sim \epsilon_M^{1/3} \approx 6 \times 10^{-6}, \quad E(h^*) \sim \epsilon_M^{2/3} \approx 4 \times 10^{-11}.$$

Two thirds of the digits at best, and only if h is chosen well (one-sided: $\epsilon_M^{1/2} \approx 10^{-8}$). And the cost: $2p$ function evaluations per gradient. For $p = 10^7$, a week of computing for one mediocre gradient.

Automatic differentiation has neither defect

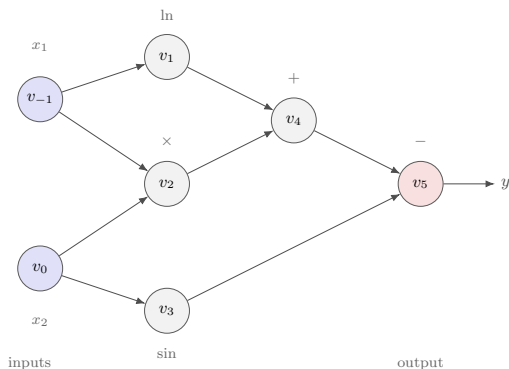
It computes the *value* of the derivative at a point (no swell), by propagating exact rules through the program (no h , no truncation), and in reverse mode obtains all p components for a small constant number of function evaluations.

The evaluation trace and the computational graph (Section 4.14.2)

AD works not on the formula for f but on the operations a program executes. For $f(x_1, x_2) = \ln x_1 + x_1 x_2 - \sin x_2$, Eq. (4.47), the **evaluation trace** (Wengert list), Eq. (4.48):

$$\begin{aligned}v_{-1} &= x_1, & v_0 &= x_2, \\v_1 &= \ln v_{-1}, & v_2 &= v_{-1} v_0, & v_3 &= \sin v_0, \\v_4 &= v_1 + v_2, & v_5 &= v_4 - v_3, & y &= v_5.\end{aligned}$$

Drawn with an edge $j \rightarrow i$ whenever v_i uses v_j : the **computational graph** (book Figure 4.5). Loops unroll, branches select – any program has a trace. A neural network is a particularly regular one. Each node's **local derivatives** are known in closed form, Eq. (4.49): $\partial v_1 / \partial v_{-1} = 1/v_{-1}$, $\partial v_2 / \partial v_{-1} = v_0$, $\partial v_2 / \partial v_0 = v_{-1}$, $\partial v_3 / \partial v_0 = \cos v_0$, the rest ± 1 . **All AD does is combine these by the chain rule. The only question is in which order.**



Forward mode: tangents (Section 4.14.3)

Fix a direction $\dot{x}$ in input space and carry with every node its **tangent**, the directional derivative

$$\dot{v}_i = \sum_k \frac{\partial v_i}{\partial x_k} \dot{x}_k, \quad \text{Eq. (4.50),} \quad \text{by the chain rule} \quad \dot{v}_i = \sum_{j \prec i} \frac{\partial \varphi_i}{\partial v_j} \dot{v}_j, \quad \text{Eq. (4.51).}$$

Runs in the *same order* as the evaluation, so each tangent is computed alongside its value.
With $\dot{x} = (1, 0)$ at $(x_1, x_2) = (2, 5)$, Eq. (4.52):

$$\begin{aligned} \dot{v}_{-1} &= 1, & \dot{v}_0 &= 0, \\ \dot{v}_1 &= \dot{v}_{-1}/v_{-1} = 0.5, & \dot{v}_2 &= \dot{v}_{-1}v_0 + v_{-1}\dot{v}_0 = 5, & \dot{v}_3 &= \cos(v_0)\dot{v}_0 = 0, \\ \dot{v}_4 &= 5.5, & \dot{v}_5 &= 5.5 = \partial f / \partial x_1. \end{aligned}$$

For $\partial f / \partial x_2$: repeat the whole sweep with $\dot{x} = (0, 1)$.

Cost

One sweep returns $\dot{y} = J\dot{x}$, a *Jacobian–vector product*, Eq. (4.55), without forming J . The gradient of a scalar function of p parameters costs p sweeps – the count of finite differences, only exact. Right when the *inputs* are few; wrong for training.

Dual numbers: forward mode in one line (Section 4.14.3)

Introduce ε with $\varepsilon^2 = 0$, $\varepsilon \neq 0$, and the **dual numbers** $a + b\varepsilon$:

$$(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon, \quad \text{Eq. (4.53),}$$

the product rule as an algebra. For any g with a Taylor series,

$$g(a + b\varepsilon) = g(a) + g'(a)b\varepsilon + \frac{1}{2}g''(a)b^2\varepsilon^2 + \cdots = g(a) + g'(a)b\varepsilon, \quad \text{Eq. (4.54).}$$

The series *terminates* – by the algebra of ε , not by neglecting anything. Evaluate g at $x + 1 \cdot \varepsilon$ and read off the coefficient of ε : $g'(x)$, exactly.

What this buys

A dual number carries a value and a derivative together. Overload the arithmetic operators of Python so that ordinary code, written with no thought of derivatives, computes them when handed a dual argument – next frame, thirty lines. This is what autograd does to numpy.

Code: a Dual class (book Section 4.14.3)

```
class Dual:
    """A dual number a + b*eps with eps**2 = 0: a value and its derivative."""
    def __init__(self, val, dot=0.0):
        self.val, self.dot = val, dot
    def _lift(o):
        return o if isinstance(o, Dual) else Dual(o)
    def __add__(self, o):
        o = Dual._lift(o); return Dual(self.val + o.val, self.dot + o.dot)
    __radd__ = __add__
    def __sub__(self, o):
        o = Dual._lift(o); return Dual(self.val - o.val, self.dot - o.dot)
    def __mul__(self, o): # the product rule, Eq. (4.53)
        o = Dual._lift(o)
        return Dual(self.val * o.val, self.val * o.dot + self.dot * o.val)
    __rmul__ = __mul__

def sin(d): return Dual(math.sin(d.val), math.cos(d.val) * d.dot) # Eq. (4.54)
def log(d): return Dual(math.log(d.val), d.dot / d.val)

def f(x1, x2): # Eq. (4.47), ordinary code
    return log(x1) + x1 * x2 - sin(x2)

print("df/dx1 =", f(Dual(2.0, 1.0), Dual(5.0, 0.0)).dot) # 1/x1 + x2 = 5.5
print("df/dx2 =", f(Dual(2.0, 0.0), Dual(5.0, 1.0)).dot) # x1 - cos(x2)
```

Reverse mode: adjoints (Section 4.14.4)

Fix the *output* instead. For scalar y define the **adjoint** of every node, $\bar{v}_i = \partial y / \partial v_i$, Eq. (4.56). Since y depends on v_j only through the nodes that consume it,

$$\bar{v}_j = \sum_{i \succ j} \bar{v}_i \frac{\partial \varphi_i}{\partial v_j}, \quad j = \ell - 1, \dots, 1 - n, \quad \text{Eq. (4.57),}$$

started from $\bar{v}_\ell = 1$: the same local derivatives, summed over *successors*, running *backwards*. At (2, 5), Eq. (4.58):

$$\begin{aligned} \bar{v}_5 &= 1, & \bar{v}_4 &= 1, & \bar{v}_3 &= -1, & \bar{v}_1 &= \bar{v}_2 = 1, \\ \bar{v}_0 &= \bar{v}_3 \cos v_0 + \bar{v}_2 v_{-1} = 2 - \cos 5 = 1.716, & \bar{v}_{-1} &= \bar{v}_2 v_0 + \bar{v}_1 / v_{-1} = 5.5. \end{aligned}$$

Both partial derivatives from *one* sweep. The two terms in $\bar{v}_0$ are the two paths from v_0 to y in the graph: the chain rule sums over paths, and the reverse sweep visits each edge once.

The price: the tape

The local derivatives need the *values* v_j ($\cos v_0$, v_{-1}), in reverse order of production. The forward evaluation must be recorded; memory $\propto$ length of the trace. This is why training a deep network is memory-bound: every activation of every layer is stored until the backward pass has consumed it.

The cheap gradient principle (Section 4.14.4)

For a vector output seeded with a covector $\bar{y}$ the reverse sweep returns $\bar{x} = J^T \bar{y}$, a *vector-Jacobian product*, Eq. (4.59) – the chain rule of Eq. (1.51) with the products taken in the cheapest order.

Proposition (Baur–Strassen, Linnainmaa)

Let $f : \mathbb{R}^p \rightarrow \mathbb{R}$ be evaluated by a trace of ℓ elementary operations with at most two arguments each. Then reverse-mode AD computes $f(x)$ and the complete gradient $\nabla f(x)$ in at most $c \ell$ operations, $c \leq 4$, **independent of p** .

Proof sketch. The forward evaluation costs ℓ and records the tape. In Eq. (4.57) each edge $j \rightarrow i$ contributes one term – one multiplication, one addition, a local derivative that is often free. At most two incoming edges per node, so at most 2ℓ edges. Nowhere did p enter: it decides how many adjoints are *read off* at the end, not how many operations are performed. $\square$ In practice $c \approx 2$ – 3 : **a gradient costs two or three function evaluations**, against $p + 1$ for finite differences and p sweeps for forward mode. For $p = 10^7$ this is the difference between a fraction of a second and a week – the entire difference between what can and cannot be trained. Backpropagation is Eq. (4.57) specialised to the layered trace of a network.

Pause and think 7: which mode for which job?

Question

(a) Training a network with $p = 10^7$ parameters and a scalar loss. (b) A network with 3 inputs (x, y, t) whose output must satisfy a partial differential equation, so you need $\partial u / \partial x$, $\partial u / \partial t$ at many points. (c) The full $m \times n$ Jacobian of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $m \approx n$. Forward or reverse?

Pause and think 7: which mode for which job?

Question

(a) Training a network with $p = 10^7$ parameters and a scalar loss. (b) A network with 3 inputs (x, y, t) whose output must satisfy a partial differential equation, so you need $\partial u / \partial x$, $\partial u / \partial t$ at many points. (c) The full $m \times n$ Jacobian of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with $m \approx n$. Forward or reverse?

Answer

Section 4.14.5 writes the Jacobian as the product $J = J_L \cdots J_1$, Eq. (4.60); forward multiplies from the right (cost $\propto n$ sweeps), reverse from the left ($\propto m$ sweeps). (a) $m = 1$, $n = 10^7$: reverse, by a factor 10^7 . (b) $n = 3$ inputs, many outputs: forward – three sweeps, and `jvp` gives each directional derivative directly (Chapter 9 uses exactly this). (c) Neither wins; the cheapest bracketing of Eq. (4.60) is NP-hard in general, use whichever and do not worry. Bonus: forward over reverse gives Hessian–vector products Hv , Eq. (4.61), for the price of a few gradients and without ever forming H – the curvature Newton’s method wanted, at a linear price.

JAX: grad, jit, vmap (Section 4.14.7)

```
import jax
jax.config.update("jax_enable_x64", True) # double precision throughout -- ALWAYS
import jax.numpy as jnp
from jax import grad, jit, vmap

def f(x):
    return jnp.sin(2 * jnp.pi * x + x**2)

df = grad(f) # reverse-mode gradient of a scalar function
print(df(1.0)) # 4.475424121402227, exact to all digits
print(grad(grad(f))(1.0)) # second derivative, by composing grad

xs = jnp.linspace(0.0, 1.0, 5)
print(vmap(df)(xs)) # the derivative at five points at once
fast_df = jit(df) # compiled on first call, then very fast
```

JAX combines the differentiation machinery with a compiler (XLA): the same code is differentiated, compiled, vectorised over a batch and run on a GPU. Forward and reverse mode are explicit as `jvp` and `vjp`, `jacfwd` and `jacrev`; `jax.hessian` is `jacfwd(jacrev(f))`.

Precision

JAX uses single precision by default. Without the first line, the gradient checks that follow agree only to seven digits, and one would wrongly conclude that AD is inexact.

The least-squares gradient without deriving it

```
Xj, yj = jnp.asarray(X), jnp.asarray(y) # the data of Section 4.4

def cost(theta, X, y): # Eq. (4.12), ordinary code
    return jnp.sum((X @ theta - y)**2) / len(y)

cost_grad = jit(grad(cost, argnums=0)) # d cost / d theta, compiled

theta = jnp.asarray(rng.normal(size=(2, 1)))
g_ad = cost_grad(theta, Xj, yj)
g_hand = (2.0 / n) * X.T @ (X @ np.asarray(theta) - y) # Eq. (4.13)
print("max |AD - hand| =", np.max(np.abs(np.asarray(g_ad) - g_hand))) # 5e-15

eta = 0.1
@jit
def gd_step(theta): # Eq. (4.15) with the automatic gradient
    return theta - eta * grad(cost)(theta, Xj, yj)
for k in range(1000):
    theta = gd_step(theta)
```

The two gradients agree to 5×10^{-15} – rounding – and the iteration returns the analytical (4.03012, 2.80580). **Do this check once, as a habit:** comparing an analytical gradient against the automatic one is the standard way of catching an error in a derivation. Project 1, part e), asks for exactly this comparison, for OLS and for Ridge.

Ridge, and a derivative nobody asked for

```
def ridge_cost(theta, X, y, lambda): # Eq. (4.16)
    return jnp.sum((X @ theta - y)**2) / len(y) + lambda * jnp.sum(theta**2)

lambda = 0.001
theta = jnp.asarray(rng.normal(size=(2, 1)))
step = jit(lambda th: th - eta * grad(ridge_cost)(th, Xj, yj, lambda))
for k in range(1000):
    theta = step(theta)

theta_closed = np.linalg.inv(X.T @ X + n * lambda * np.eye(2)) @ X.T @ y # the n again
print("gradient descent:", np.asarray(theta).ravel())
print("closed form: ", theta_closed.ravel())

# the derivative with respect to lambda, argument number 3
dC_dlambda = grad(ridge_cost, argnums=3)(theta, Xj, yj, lambda)
print(dC_dlambda, "=", jnp.sum(theta**2)) # theta^T theta, by Eq. (4.16)
```

Changing the cost changes *one* function; the gradient follows. And grad differentiates with respect to *any* argument: $\partial C / \partial \lambda = \theta^T \theta$ falls out for free – the kind of quantity one wants when tuning hyperparameters by gradient methods rather than by grid search.

Hessians, one Newton step, and Hessian–vector products

```
from jax import hessian, jvp

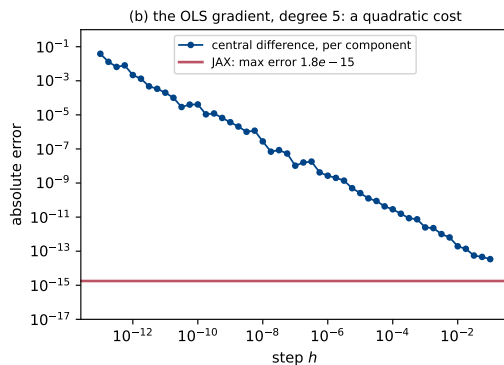
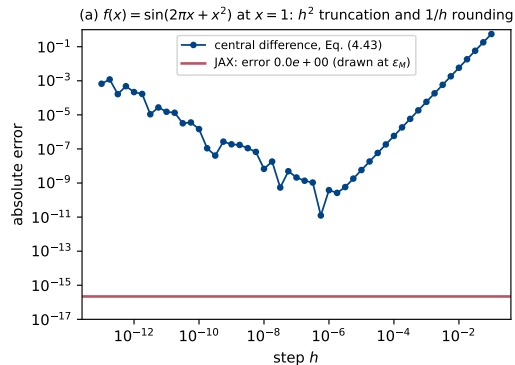
H = hessian(cost)(theta, Xj, yj).reshape(2, 2) # forward-over-reverse
print(H); print((2.0 / n) * X.T @ X) # Eq. (4.14), to rounding

theta0 = jnp.asarray(rng.normal(size=(2, 1))) # one Newton step, Eq. (4.8)
theta_newton = theta0 - jnp.linalg.solve(H, grad(cost)(theta0, Xj, yj))
print("one Newton step:", np.asarray(theta_newton).ravel()) # the exact minimum

def hvp(f, theta, v): # Eq. (4.61): H v without H
    return jvp(grad(f), (theta,), (v,))[1]
```

Everything this morning promised is now a few lines: the Hessian to rounding, the single exact Newton step of Eq. (4.9), and – for the problems where H cannot be stored – Hv for the price of a few gradient evaluations. Book Section 4.14.8 runs gradient descent, momentum, Adam and Newton on the non-convex Rosenbrock function with nothing derived by hand: 27 206, 2 591, 2 934 and 6 iterations. Replace the function by a network with a million parameters and only its definition changes.

Finite differences against automatic differentiation, measured



(a) $f(x) = \sin(2\pi x + x^2)$ at $x = 1$, book Section 4.14.9: the error falls as h^2 until $h \approx 10^{-6}$, then rises as $1/h$ – best 10^{-11} . JAX returns 4.475424121402227, every digit correct. **(b)** The degree-5 OLS gradient of this week's exercises, one central difference per component: the JAX gradient agrees with Eq. (4.13) to 1.8×10^{-15} for any h , because it involves no h . Whenever you use a finite-difference check (Eq. 4.63), h must sit at the bottom of the valley – and agreement better than 10^{-8} should not be expected.

Pause and think 8: the missing branch

Question

Panel (b) has no rising h^2 branch on the right: the finite difference of the OLS gradient is *best* at $h = 0.1$, with error 3×10^{-14} . Why – and does that make finite differences good enough for Project 1?

Pause and think 8: the missing branch

Question

Panel (b) has no rising h^2 branch on the right: the finite difference of the OLS gradient is *best* at $h = 0.1$, with error 3×10^{-14} . Why – and does that make finite differences good enough for Project 1?

Answer

The OLS cost is a quadratic, so $f''' \equiv 0$ and the truncation term of Eq. (4.45) vanishes identically: a central difference of a quadratic is exact for *any* h , and only rounding, $\propto 1/h$, remains. Ridge is quadratic too. So yes, for parts a)–f) finite differences would pass the check – and would still cost $2p$ cost evaluations per gradient against two or three for reverse mode, and would fail the moment the cost stops being quadratic (logistic regression next week: panel (a) is the general case). Use the finite difference as a *check* at $h \approx 10^{-5}$; use AD, or the analytical gradient, for the descent.

Summary of week 37

- ▶ **Cross-validation** chooses the hyperparameter (Eqs. 3.92–3.96): leak-free folds, $CV \pm SE$, $\lambda_{\min}$ or one-SE, refit, test set once. The bootstrap says which coefficients to believe.
- ▶ Convex cost $\Rightarrow$ any stationary point is the global minimum (Eq. 4.2). Newton's method, Eq. (4.8), is the ideal: one exact step on OLS, but $\mathcal{O}(p^3)$.
- ▶ Gradient descent, Eq. (4.15), replaces H^{-1} by a scalar η . The error decouples along eigendirections, Eq. (4.19): stable iff $\eta < 2/\lambda_{\max}$ (Eq. 4.20), iterations $\propto \kappa = \lambda_{\max}/\lambda_{\min}$ (Eq. 4.21), and for regression $\kappa(H) = \kappa_2(X)^2$ (Eq. 4.22): standardise, and note that Ridge conditions the problem.
- ▶ Momentum, Eq. (4.23), amplifies consistent gradients by $1/(1 - \gamma)$ and cancels oscillating ones: $\kappa \rightarrow \sqrt{\kappa}$ (Eq. 4.26). A factor 11 on the Runge problem from one line.
- ▶ Automatic differentiation propagates exact derivative rules through the evaluation trace: forward mode (tangents, dual numbers), reverse mode (adjoints, the tape), and the cheap gradient principle – a gradient for the price of two or three function evaluations, independent of p . `jax.grad` of your cost function *is* your gradient; check it once against Eq. (4.13).

Tomorrow, the deadline, and next week

Tuesday: Case 1 = your own gradient descent for OLS and Ridge with analytical gradients, learning rates and stopping; Case 2 = momentum and the JAX gradient on the Runge function of Project 1. Exercises due **Friday September 11 at midnight**. Week 38: stochastic gradient descent, AdaGrad, RMSProp and Adam, applied to Lasso and logistic regression.